CS 498: Machine Perception
HW3: Center-Based 3D Detection from LiDAR

Fall 2026

Course weight	15%
Work mode	Individual
Release	Friday, October 23, 2026
Due	Friday, November 6, 2026 at 11:59 p.m. America/Chicago

1 What you will build

A self-driving system does not receive a list of nearby vehicles. It receives a sparse LiDAR sweep: tens of thousands of 3D points with no object identity. In this homework, you will turn one sweep into oriented 3D vehicle boxes using a compact center-based detector. You will implement the complete path:

$$\text{LiDAR points} \rightarrow \text{BEV tensor} \rightarrow \text{center targets} \rightarrow \text{dense CNN heads} \rightarrow \text{metric 3D boxes.}$$

The design follows CenterPoint: objects are detected by their centers on a bird's-eye-view grid, while separate heads regress the remaining box attributes. The assignment uses one class, `Car`, so the focus stays on representation, target construction, losses, decoding, and evaluation rather than a large taxonomy.

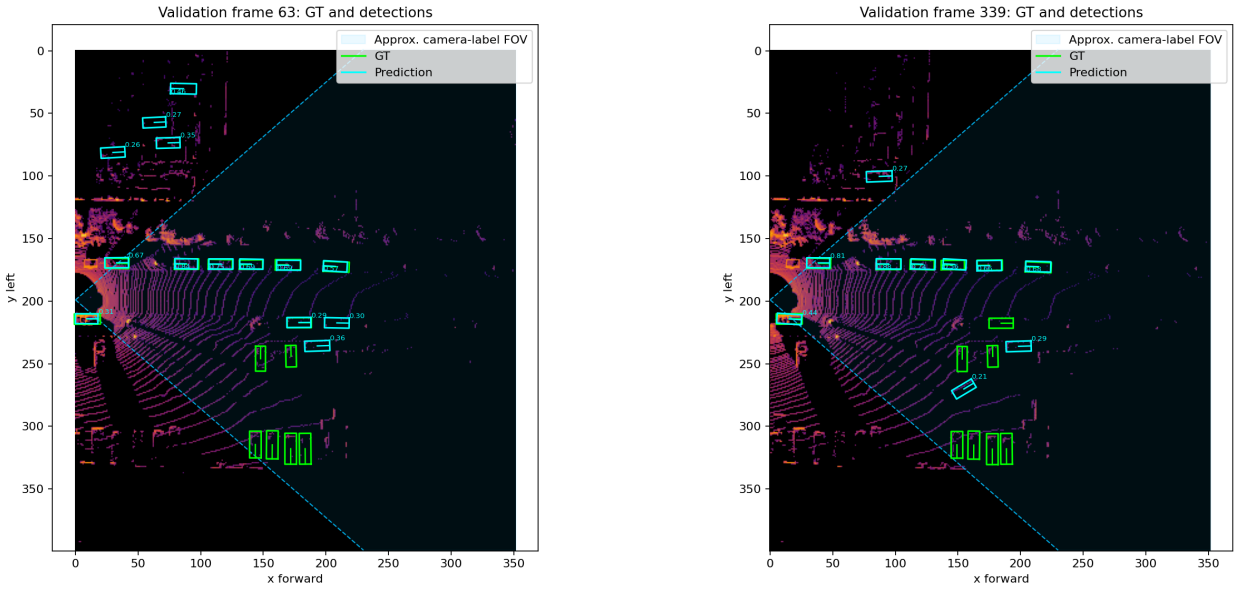


Figure 1: Desired detector output. Green boxes are ground truth; cyan boxes are decoded predictions with confidence scores. The translucent blue region is the approximate front-camera label FOV.

2 Run one file per task

Install the small dependency list from the `handout/` directory, then run the quick debug pipelines:

```
python -m pip install -r requirements.txt
```

```
python task1_bev.py
python task2_targets.py
python task3_detector.py --debug
python task4_inference.py --debug
```

Edit only code between "Modify your code below" and "DO NOT MODIFY CODE OUTSIDE THE BLOCK." Each file contains its full sequential `main()` and writes a named checkpoint. The supplied real-KITTI debug archives support this two-epoch smoke run only; it is far too small for a meaningful accuracy result.

The full release uses all 7,481 labeled training frames from the KITTI 3D object-detection benchmark. Its `train.npz`, `val.npz`, and `test.npz` archives contain 5,977, 730, and 774 frames from 123, 8, and 10 raw drives. KITTI's official object-to-raw mapping keeps all frames from each drive in one split. The drive-level assignment also balances empty frames, object range, occlusion, 2D box size, and capture date. Download and verify the separately supplied `hw3_data.zip` package from the public course dataset repository with:

```
python data/download_data.py
python task3_detector.py
python task4_inference.py
```

This creates the three NPZ files in `handout/data/`; the large archives are not stored in Git. The default training and validation commands require the full data and a matching trained checkpoint; missing prerequisites cause an error rather than a silent debug or untrained-model fallback. Use training data to fit parameters, validation data for every checkpoint and score-threshold decision, and test data only once after all choices are frozen. The selected threshold records its validation source and checkpoint hash; `--test` checks both before reading test data and rejects a repeat run. Full training is GPU-oriented. On the staff RTX 4080, batch size 16 used about 11.4 GiB; the default batch size 8 used about 6.7 GiB. The `--batch-size4` option trades speed for lower memory. The explicit `--debug` smoke run also works on CPU.

KITTI labels objects visible in the front color camera, although each Velodyne sweep covers 360 degrees. The translucent blue wedge in our BEV figures marks the approximate camera-label field of view. Returns outside this wedge can be real objects without corresponding benchmark labels; the exact image boundary varies slightly with calibration.

Run the final test exactly once with:

```
python task4_inference.py --test
```

File	Graded edits
`task1_bev.py`	`rasterize_points`
`task2_targets.py`	`encode_targets`, `decode_targets`
`task3_detector.py`	`CenterDetector.forward`, `focal_loss`, `regression_l1`
`task4_inference.py`	`suppress_nonmaxima`, `decode_predictions`

3 Representation contract

We use the LiDAR coordinate frame throughout: x points forward, y points left, and z points up. Each oriented box is

$$\mathbf{b} = [x, y, z, l, w, h, \theta],$$

where (x, y, z) is its center, (l, w, h) are metric dimensions, and θ is yaw around the z axis.

The region of interest and discretization are fixed:

$$x \in [0, 70.4), \quad y \in [-40, 40), \quad z \in [-3, 1], \quad \Delta = 0.2 \text{ m}, \quad s = 4.$$

Thus the model input is `[B,4,400,352]`, and every output head has spatial shape `[B,*,100,88]`. One output cell covers 0.8 m by 0.8 m. For metric point (x, y), use

$$c = \left\lfloor \frac{x - x_{\min}}{\Delta} \right\rfloor, \qquad r = (H - 1) - \left\lfloor \frac{y - y_{\min}}{\Delta} \right\rfloor.$$

The reversed row coordinate makes the displayed BEV use y-left consistently.

4 Task 1: BEV rasterization [2.0 points]

Implement `rasterize_points(points)`. Input has shape `[N,4]` with $(x, y, z, \text{intensity})$. Output is float32 `[4,400,352]` in $[0, 1]$, with channels in this exact order:

Channel	Definition
Max height	maximum z in the cell, normalized from $[-3, 1]$ to $[0, 1]$
Mean height	mean z in the cell, normalized in the same way
Intensity	mean reflectance, clipped to $[0, 1]$
Density	$\log(1 + n)/\log(65)$, clipped to $[0, 1]$

Filter points outside the ROI. Use `metric_to_grid`, flatten each valid cell as `row*GRID_W+column`, use `np.bincount` for counts and sums, and `np.maximum.at` for maxima. Do not loop over BEV cells. Empty cells must be exactly zero in all channels.

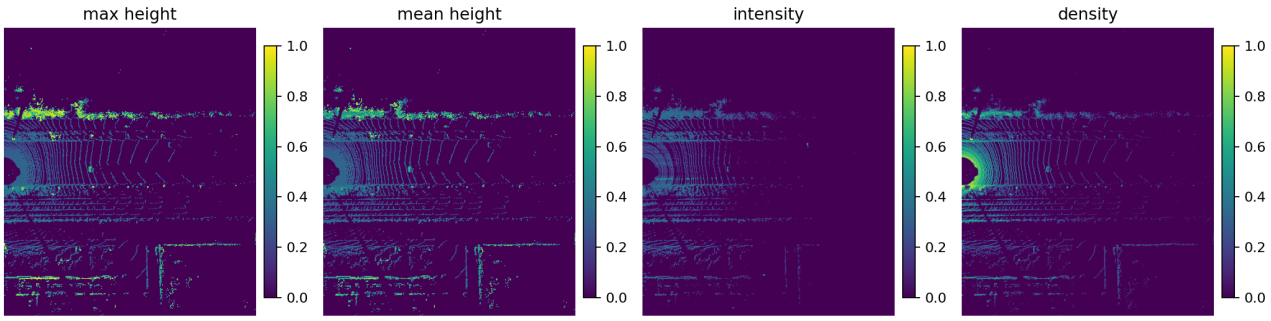


Figure 2: Reference BEV channels for one real KITTI sweep. A correct solution has visible road and object structure rather than empty, saturated, mirrored, or transposed maps.

Checkpoint 1 writes `bev_channels.png`, `bev_boxes.png`, and `metrics.json`. Confirm shape `[4,400,352]`, finite values in $[0, 1]$, occupancy around 5–10% on the debug frame, and vehicle boxes aligned with occupied structure.

5 Task 2: Center targets [3.0 points]

Now convert metric boxes into the dense and sparse supervision consumed by the detector. For a box center, its continuous stride-4 grid location is

$$u = \frac{x - x_{\min}}{s\Delta}, \qquad v = H_{\text{out}} - \frac{y - y_{\min}}{s\Delta}.$$

The integer cell is $(\lfloor u \rfloor, \lfloor v \rfloor)$ and the sub-cell offset is $(u - \lfloor u \rfloor, v - \lfloor v \rfloor)$. Use the supplied `metric_to_output_grid` and `output_grid_to_metric` helpers; they also resolve the exact boundaries of the half-open ROI so target cells align with the stride-4 BEV cells.

Implement `encode_targets`. Draw a Gaussian around each valid center on the `[1,100,88]` heatmap. At its discrete center slot, store the flat index, sub-cell offset, metric center height, $(\log l, \log w, \log h)$, and $(\sin\theta, \cos\theta)$. Set the object mask. Regression is supervised only at marked centers; the heatmap is spatially dense.

Implement `decode_targets` as the exact inverse. Recover continuous (u, v), convert it to metric (x, y), exponentiate dimensions, and use $\text{atan2}(\sin\theta, \cos\theta)$ for yaw.

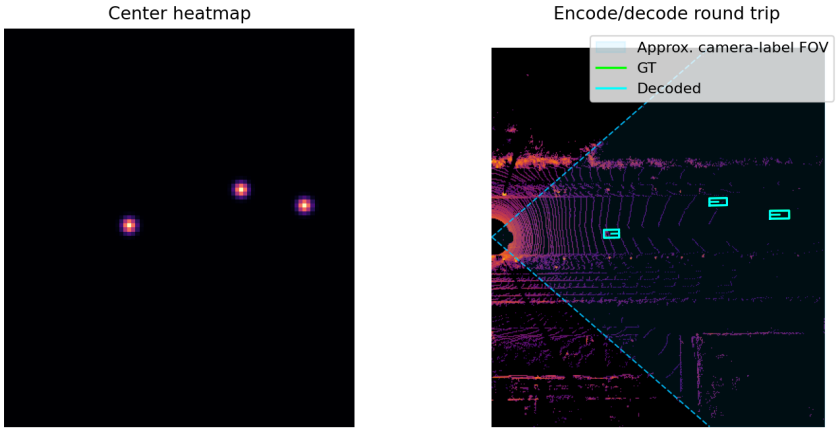


Figure 3: Reference center heatmap and metric encode/decode round trip. Green ground truth and cyan decoded boxes overlap.

Checkpoint 2 writes `target_roundtrip.png` and `metrics.json`. The heatmap maximum should be 1, encoded/decoded counts should agree with valid ground truth, and mean center error should be below 10^{-4} m.

6 Task 3: Detector and losses [6.0 points]

The fixed network maps `[B,4,400,352]` to a stride-4 feature map `[B,128,100,88]`. Two stride-2 residual blocks provide the spatial downsampling. Each already-declared head is a 3×3 convolution, ReLU, and 1×1 convolution.

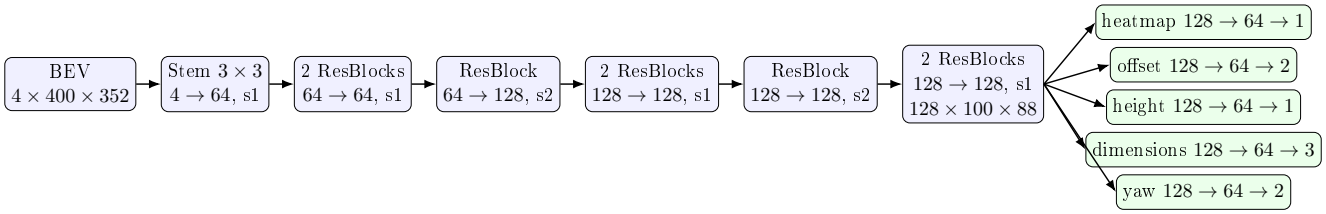


Figure 4: Required compact center-based detector. "s1" and "s2" denote stride. The module definitions are supplied; you implement the visible forward order.

Implement `CenterDetector.forward` by applying the declared modules in the documented order and returning all five heads. Do not add or resize layers.

Implement the CenterNet heatmap focal loss:

$$\mathcal{L}_{hm} = -\frac{1}{\max(N_{pos}, 1)} \sum_i \begin{cases} (1-p_i)^2 \log p_i, & H_i = 1, \\ (1-H_i)^4 p_i^2 \log(1-p_i), & H_i < 1. \end{cases}$$

When there are no positive centers, return only the summed negative term.

Implement `regression_l1`. Use `gather_map` to gather a dense prediction map at the target center indices, apply the object mask, sum absolute error, and divide by the number of valid objects. Do not divide by the number of channels.

The provided pipeline trains for at most 20 epochs on the final split, evaluates on validation every epoch, stops after validation AP@0.5 fails to improve for eight epochs, and preserves `detector.pt` at the highest validation AP@0.5. It writes `training_curves.png` and `metrics.json`. Report the full loss behavior and validation AP rather than debug-split accuracy.

7 Task 4: Decode and evaluate [2.5 points]

Implement `suppress_nonmaxima`: max-pool the sigmoid heatmap with kernel 3, stride 1, and padding 1, then keep only pixels equal to their local maximum.

Implement `decode_predictions`. Use the provided top-k helper, gather all four regression heads, add offsets to grid centers, and convert back to metric coordinates:

$$x = x_{\min} + u(s\Delta), \qquad y = y_{\min} + (H_{out} - v)(s\Delta).$$

Exponentiate dimensions, recover yaw with `atan2`, filter by confidence, and apply the provided rotated-IoU NMS. Preserve one dictionary of boxes, scores, and classes per batch item.

The validation pipeline computes AP from the full ranked prediction list, selects one displayed operating threshold by F1, writes it to `selected_threshold.json`, produces two fixed GT/prediction figures and a PR curve, and reports operating-point precision/recall and AP at rotated IoU 0.5. Freeze the checkpoint and threshold before the single final `--test` run. Discuss both scalar AP and the shape of the PR curve. This course split and metric do not use KITTI's official difficulty protocol, so do not compare the resulting AP directly with leaderboard values.

8 Submission and report [1.5 points]

Submit exactly:

- revised `task1_bev.py`;
- revised `task2_targets.py`;
- revised `task3_detector.py`;
- revised `task4_inference.py`; and
- one report PDF following `report_template.tex`.

Do not submit datasets, checkpoints, or the full output directory. Every report figure must be reproduced by submitted code. Keep validation visualizations fixed across comparisons and include the single final test aggregate result.

The required score totals 15.0 points: Task 1 (2.0), Task 2 (3.0), Task 3 (6.0), and Task 4 (2.5), plus the report and reproducibility (1.5).

9 Optional extensions

Gen-AI leverage allowed

Tracking with data association [+2.0 points]. Use the supplied `tracking_sequence.npz`, which contains 154 consecutive frames from KITTI raw drive `2011_09_26_drive_0005_sync`. Run your frozen detector and validation-selected score threshold on every frame. Ground-truth boxes and track IDs are reserved for final evaluation and comparison visualization; do not use them for association, filtering, tuning, or track birth/death decisions.

Implement `bonus_tracking.py` yourself; no tracker, association, metric, or rendering helper is supplied. Maintain a constant-velocity Kalman state for each object, predict before association, use Hungarian matching with a rotated-BEV-IoU-based cost and a reasonable gate, update matched tracks, create tracks from unmatched detections, and retire stale tracks. Render all frames in temporal order. Every predicted track must display its ID and retain the same distinct color throughout the video.

Evaluate at rotated BEV IoU 0.5 and report ID switches, MOTA, MOTP, and IDF1:

$$\mathrm{MOTA} = 1 - \frac{\mathrm{FN} + \mathrm{FP} + \mathrm{IDSW}}{\mathrm{GT}}, \qquad \mathrm{MOTP} = \frac{\sum_{\text{matches}} \mathrm{IoU}}{\#\text{matches}},$$

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}}.$$

Use this matching protocol so results are comparable. At each frame, preserve a previous-frame GT/prediction assignment if its current IoU remains at least 0.5, then use Hungarian matching on the remaining boxes with IoU at least 0.5. Count an ID switch when a previously matched GT identity is next matched to a different predicted identity, including after a gap. For IDF1, count valid IoU matches over the whole sequence for every GT/predicted identity pair and use one global Hungarian assignment that maximizes that count. Its sum is IDTP; total predictions minus IDTP is IDFP, and total GT boxes minus IDTP is IDFN.

Here MOTP is mean matched IoU, so higher is better. Run `bonus_tracking.py` with Python. Encoding the required MP4 also needs the `ffmpeg` command-line program. Submit `bonus_tracking.py`, a bonus report section containing the four metrics, tracker parameters and one analyzed failure case, and an MP4 named `tracking_comparison.mp4`. Do not submit the dataset.

Multi-class detection [+1.0 point]. The required split archives contain only Car labels. First download the small, public companion after the core package by running `python data/download_multiclass_data.py`. Its `multiclass_train.npz`, `multiclass_val.npz`, and `multiclass_test.npz` contain class IDs and LiDAR boxes aligned by frame ID and row order to the core BEV arrays; see `data/README.md` for keys and counts. This optional package does not alter the required detector or its data. Implement `bonus_multiclass.py` yourself. Add Pedestrian and Cyclist consistently through three-class targets and heatmaps, decoding, class-wise NMS, visualization, and per-class PR/AP at rotated BEV IoU 0.5. Submit code, per-class validation PR/AP, at least two fixed qualitative validation views, and a bonus report section. Choose checkpoints and thresholds on validation only. If you report a final test metric, run it once after freezing those choices; the test split contains relatively few Cyclist examples, so discuss that uncertainty. No multiclass helper implementation is supplied.

10 Integrity and permitted tool use

For the required assignment, you are not expected to use generative AI coding or writing tools such as Copilot, Codex, Claude, or ChatGPT. AI may be used only for grammar and typo checking in the required write-up. AI-assisted techniques are allowed only for the optional extensions explicitly marked above.

Stronger checks include short code walkthroughs, version history, submission-specific questions, and consistency among the report, code, and outputs. You should be able to explain the tensor shape and purpose of every line inside your edit blocks and reproduce every submitted figure from your code.